DFS序维护树上信息

1 核心原理：DFS 序及其性质

DFS 序是指在对树进行深度优先遍历时，节点被访问的先后顺序。

- **性质**：一棵子树中所有节点的 DFS 序是一个**连续的区间**。
- **记录方式**：
  - `st[u]`：进入节点 u 的时间戳（区间左端点）。
  - `en[u]`：离开节点 u 时的最大时间戳（区间右端点）。
  - 子树 u 的范围即为：$[st[u], en[u]]$。

2 模型一：点修改，子树查询

将点修改转化为 DFS 序上的单点修改，子树查询转化为区间查询。

2.1 步骤

对于任意节点 u，其子树中所有节点的 DFS 序（即 st 值）都落在闭区间 $[st[u], en[u]]$ 之内。

当我们查询以 x 为根的**子树点权和**时，操作步骤如下：

$$\text{SubtreeSum}(x) = \sum_{i=st[x]}^{en[x]} \text{Value}(\text{node at DFS position } i) \tag{51}$$

为了支持快速修改和查询，我们通常在 DFS 序上建立**树状数组 (BIT)** 或 **线段树**。

- **修改**：当节点 x 的权值增加 y 时，对应在树状数组的 $st[x]$ 位置执行单点更新。
- 查询：查询子树和即执行树状数组的区间查询操作。根据前缀和原理：

$$\text{SubtreeSum}(x) = \text{Query}(en[x]) - \text{Query}(st[x] - 1) \tag{52}$$

2.2 复杂度分析

- **预处理**：一次 DFS 遍历，时间复杂度 $O(n)$。
- **点修改**：树状数组单点更新，时间复杂度 $O(\log n)$。
- **子树查询**：树状数组区间求和，时间复杂度 $O(\log n)$。

2.3 标准代码

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 1e6 + 5;

int n, m, p, cnt, tim;
int h[N], st[N], en[N];
ll a[N], s[N];

struct edge { int v, nxt; } e[N << 1];

void add(int u, int v) {
    e[++cnt] = {v, h[u]};
    h[u] = cnt;
}

void dfs(int u, int fat) {
    st[u] = ++tim;
    for (int i = h[u]; i; i = e[i].nxt) {
        if (e[i].v != fat) dfs(e[i].v, u);
    }
    en[u] = tim;
}

void update(int x, ll y) {
    for (; x <= n; x += x & -x) s[x] += y;
}

ll query(int x) {
    ll res = 0;
    for (; x; x -= x & -x) res += s[x];
    return res;
}

int main() {
    scanf("%d%d%d", &n, &m, &p);
    for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
    for (int i = 1; i < n; i++) {
        int u, v; scanf("%d%d", &u, &v);
        add(u, v); add(v, u);
    }
    dfs(p, 0);
    for (int i = 1; i <= n; i++) update(st[i], a[i]);
    while (m--) {
        int op, x; ll y;
        scanf("%d", &op);
        if (op == 1) {
```

```
            scanf("%d%lld", &x, &y);
            update(st[x], y);
        } else {
            scanf("%d", &x);
            printf("%lld\n", query(en[x]) - query(st[x] - 1));
        }
    }
    return 0;
}
```

3 模型二 & 三：树链修改，点/子树查询

模型二与模型三的核心逻辑是将**路径（树链）上的修改**转化为**节点的差分标记**，从而利用 DFS 序将复杂的树上求和问题降维成线性的区间问题。

3.1 步骤

要在路径 (u, v) 上的所有节点增加 w，如果直接遍历路径，时间复杂度最高为 $O(n)$。利用差分思想，我们可以只在四个关键节点上进行标记。

我们定义一个差分数组 d。

一个点 i 的当前真实权值 A_i 定义为：以 i 为根的子树内所有 d 值的总和。

$$A_i = \sum_{j \in \text{subtree}(i)} d_j \tag{53}$$

为了让路径 (u, v) 上的 A_i 增加 w，我们在树状数组（维护在 DFS 序上）中执行：

1. $d[st[u]] \leftarrow d[st[u]] + w$
2. $d[st[v]] \leftarrow d[st[v]] + w$
3. $d[st[LCA(u, v)]] \leftarrow d[st[LCA(u, v)]] - w$
4. $d[st[fa[LCA(u, v)]]] \leftarrow d[st[fa[LCA(u, v)]]] - w$

逻辑解释：

- 一个修改点 j 对其祖先节点 i 的 A_i 有贡献。
- u 的修改对 $u \to root$ 路径有贡献；v 的修改对 $v \to root$ 路径有贡献。
- 这样 $LCA \to root$ 被多算了两遍，通过在 LCA 和 $fa[LCA]$ 减去 w 来抵消，最终只有路径 (u, v) 被覆盖。

3.2 模型二：点查询

在执行完上述差分修改后，查询某个点 x 的权值就变成了查询其子树的差分值之和。

在 DFS 序中，子树 x 对应的区间是 $[st[x], en[x]]$。

- **计算公式：** $A_x = \text{sum}(en[x]) - \text{sum}(st[x]-1) + \text{InitialValue}_x$
- 使用**树状数组**维护 DFS 序，即可在 $O(\log n)$ 内完成查询。

3.3 模型三：子树查询

如果我们要查询以 x 为根的**整棵子树**的权值和（即 $\sum_{i \in \text{subtree}(x)} A_i$），需要推导差分标记的贡献。

假设我们在节点 y 处有一个差分值 w（y 是 x 子树内的某个点）。这个 w 会对 y 及其直到 x 的所有祖先产生贡献。

对于节点 x 的子树和，这个 w 产生的总贡献次数是：从 y 向上到 x 的节点个数，即 $(deep[y] - deep[x] + 1)$。

总贡献公式：

$$\text{Contribution} = \sum_{y \in \text{subtree}(x)} (w_y \times (deep[y] - deep[x] + 1)) \tag{54}$$

拆开括号：

$$\text{Contribution} = \sum (w_y \times deep[y]) - \sum (w_y \times (deep[x] - 1)) \tag{55}$$

由于对于特定的查询节点 x，$(deep[x] - 1)$ 是一个常数，可以提到求和符号外面：

$$\text{Contribution} = \underbrace{\sum_{y \in \text{subtree}(x)} (w_y \times deep[y])}_{\text{数组 } S_2} - (deep[x] - 1) \times \underbrace{\sum_{y \in \text{subtree}(x)} w_y}_{\text{数组 } S_1} \tag{56}$$

我们需要在 DFS 序上建立**两个**树状数组：

1. S_1 **(差分数组)**：维护每个点的 w。
2. S_2 **(乘积数组)**：维护每个点的 $w \times deep[y]$。

查询 x 子树和的步骤：

1. 计算 S_2 在区间 $[st[x], en[x]]$ 的区间和。
2. 计算 S_1 在区间 $[st[x], en[x]]$ 的区间和，并乘以 $(deep[x] - 1)$。
3. 两者相减，最后加上原有的初始子树和 $ss[x]$。

3.4 时间复杂度

操作	复杂度	关键点

操作	复杂度	关键点
路径修改	$O(\log n + \mathrm{LCA})$	四点差分标记，更新两个 BIT
点查询	$O(\log n)$	子树差分值求和
子树查询	$O(\log n)$	利用两个 BIT 进行深度加权求和

3.5 标准代码

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 1e6 + 5;

int n, m, r, tim, cnt;
int h[N], st[N], en[N], dep[N], f[N][21], lg[N];
ll a[N], ss[N], s1[N], s2[N];
struct edge { int v, nxt; } e[N << 1];


void add(int u, int v) { e[++cnt] = {v, h[u]}; h[u] = cnt; }

void dfs(int u, int fat) {
    st[u] = ++tim;
    dep[u] = dep[fat] + 1;
    f[u][0] = fat;
    ss[u] = a[u];
    for (int i = 1; i <= lg[dep[u]]; i++) f[u][i] = f[f[u][i - 1]][i - 1];
    for (int i = h[u]; i; i = e[i].nxt) {
        if (e[i].v != fat) {
            dfs(e[i].v, u);
            ss[u] += ss[e[i].v];
        }
    }
    en[u] = tim;
}

int get_lca(int x, int y) {
    if (dep[x] < dep[y]) swap(x, y);
    while (dep[x] > dep[y]) x = f[x][lg[dep[x] - dep[y]] - 1];
    if (x == y) return x;
    for (int i = lg[dep[x]] - 1; i >= 0; i--)
        if (f[x][i] != f[y][i]) x = f[x][i], y = f[y][i];
    return f[x][0];
}

void bit_upd(int x, ll v1, ll v2) {
    for (; x <= n; x += x & -x) s1[x] += v1, s2[x] += v2;
}
```

```c

void modify(int x, ll v) {
    if (!x) return;
    bit_upd(st[x], v, v * dep[x]);
}

ll bit_qry(int x, ll s[]) {
    ll res = 0;
    for (; x; x -= x & -x) res += s[x];
    return res;
}

ll get_sum(int x, ll s[]) { return bit_qry(en[x], s) - bit_qry(st[x] - 1, s); }

int main() {
    scanf("%d%d%d", &n, &m, &r);
    for (int i = 1; i <= n; i++) scanf("%lld", &a[i]);
    for (int i = 1; i < n; i++) {
        int u, v; scanf("%d%d", &u, &v);
        add(u, v); add(v, u);
    }
    for (int i = 1; i <= n; i++) lg[i] = lg[i - 1] + (1 << lg[i - 1] == i);
    dfs(r, 0);
    while (m--) {
        int op, u, v; ll w;
        scanf("%d", &op);
        if (op == 1) {
            scanf("%d%d%lld", &u, &v, &w);
            int lca = get_lca(u, v);
            modify(u, w); modify(v, w); modify(lca, -w); modify(f[lca][0], -w);
        } else if (op == 2) {
            scanf("%d", &u);
            printf("%lld\n", get_sum(u, s1) + a[u]);
        } else {
            scanf("%d", &u);
            printf("%lld\n", get_sum(u, s2) - get_sum(u, s1) * (dep[u] - 1) + ss[u]);
        }
    }
    return 0;
}
```

---

4 模型四：点修改，链和查询

4 模型四：点修改，链和查询

4.1 步骤

假设树上一个点 y 的点权增加了 v。

- 对于任何节点 x，如果 y 是 x 的祖先（即 x 在 y 的子树内），那么从 $root$ 到 x 的路径上必然包含 y。
- 因此，$root \to x$ 的路径和会因为 y 的增加而同步增加 v。
- 如果 y 不是 x 的祖先，则 $root \to x$ 的路径和不受影响。

结论： 对点 y 的**单点修改**，等价于对以 y 为根的**整棵子树**内所有节点的"路径和"进行**区间增加**。

利用 DFS 序的性质，子树 y 对应线性区间 $[st[y], en[y]]$。

我们要维护的是每个节点到根的路径和 $Dis(x)$。当点 y 权值增加 v 时：

- 所有满足 $i \in [st[y], en[y]]$ 的节点，$Dis(i)$ 都要加上 v。
- 这变成了一个典型的**区间修改、单点查询**问题。

为了高效实现，我们使用**树状数组维护差分**：

1. 在 $st[y]$ 位置加上 v。
2. 在 $en[y]+1$ 位置减去 v。
3. 查询某个点 a 当前的路径和增量时，只需要求树状数组的前缀和 $\sum_{i=1}^{st[a]} s[i]$。

题目要求查询任意两个节点 a, b 之间的路径和。根据树上路径的性质，路径 (a, b) 的和可以拆解为：

$$\text{PathSum}(a, b) = Dis(a) + Dis(b) - Dis(LCA) - Dis(fa[LCA]) \tag{57}$$

4.2 时间复杂度

步骤	操作内容	时间复杂度
预处理	DFS 求出 st, en、深度、LCA 倍增表及初始路径和 `size[x]`	$O(n \log n)$
点修改 (y, v)	树状数组执行 `update(st[y], v)` 和 `update(en[y]+1, -v)`	$O(\log n)$
链和查询 (a, b)	求出 LCA 后，计算四个点的 Dis 值并按照公式合并	$O(\log n)$

4.3 标准代码

```
if (op == 1) { // 点修改
    int x, y; scanf("%d%d", &x, &y);
    ad(be[x], y); ad(en[x] + 1, -y); // 差分维护
} else { // 链查询
    int a, b; scanf("%d%d", &a, &b);
    int lca = LCA(a, b);
```

```
    ll ans = (size[a] + size[b] - size[lca] - size[f[lca][0]]) +
             (sum(be[a]) + sum(be[b]) - sum(be[lca]) - sum(be[f[lca][0]]));
    printf("%lld\n", ans);
}
```

---

5 模型五 & 六：子树修改，点/子树查询

5.1 模型五：子树修改，点查询

虽然这是一个“区间修改，单点查询”的问题，但通常有两种实现方案：

5.1.1 方案 A：差分 + 树状数组（推荐）

这是最轻量级的做法。

- **修改**：在 $st[u]$ 处 $+v$，在 $en[u]+1$ 处 $-v$。
- **查询**：查询点 x 的权值即为差分数组的前缀和 $\sum_{i=1}^{st[x]} d[i]$。加上初始权值即得答案。

5.1.2 方案 B：线段树

使用线段树维护，支持区间加和单点查询（查询区间 $[st[x], st[x]]$）。对于大规模数据，线段树常数略大，但通用性更强。

6 模型六：子树修改，子树查询

这是最标准的“区间修改，区间查询”问题，通常使用**线段树**配合 **Lazy Tag（懒标记）** 来实现，以保证 $O(\log n)$ 的操作效率。

6.1 线段树维护步骤

1. **建树 (Build)**：根据 DFS 序，将原始点权填入数组。注意：数组下标 i 存储的是 `dfn[i]` 节点的权值。
2. **区间修改 (Update)**：
   - 目标区间：$[st[x], en[x]]$。
   - 逻辑：若当前区间被目标区间覆盖，则更新当前节点的 `dat`（值）并打上 `laz`（标记）。
3. **区间查询 (Query)**：
   - 目标区间：$[st[x], en[x]]$。
   - 逻辑：递归下传懒标记，累加区间内的 `dat`。

注意：

- **数组映射关系**：在建树或初始赋值时，必须明确 `a[st[u]] = val[u]`。即：树状数组/线段树的第 i 个位置，存放的是 DFS 序为 i 的那个节点的权值。
- **区间边界**：子树的范围是 $[st[x], en[x]]$，其中 $en[x]$ 是在 DFS 退出该节点时记录的最大时间戳，加

- **区间边界**：子树的范围是 $[st[x], en[x]]$，其中 $en[x]$ 是在 DFS 退出该节点时记录的最大时间戳。如果 DFS 实现不同（如只记录入序），区间长度通常为 `st[x] + size[x] - 1`。
- **空间开销**：线段树通常需要开 $4N$ 的空间，如果是点数达到 10^6 的题目，需注意内存限制。

6.2 复杂度分析

操作类型	时间复杂度	空间复杂度
预处理 (DFS)	$O(n)$	$O(n)$
子树修改	$O(\log n)$	$O(n)$ 或 $O(4n)$
子树查询	$O(\log n)$	$O(n)$ 或 $O(4n)$

6.3 标准代码

```cpp
// 线段树维护子树
void change(int p, int l, int r, int lr, int rr, ll v) {
    if (lr <= l && r <= rr) {
        lazy(p, v);
        return;
    }
    pushdown(p);
    int mid = (l + r) >> 1;
    if (lr <= mid) change(p << 1, l, mid, lr, rr, v);
    if (rr > mid) change(p << 1 | 1, mid + 1, r, lr, rr, v);
    update(p);
}
// 调用时：change(1, 1, n, be[x], en[x], v);
```

---

7 模型七：子树修改，链查询

7.1 步骤

假设我们给以节点 y 为根的子树中所有点权增加了 v。

现在我们要查询 $root \to x$ 的链和。这个修改会对链和产生贡献，当且仅当 y 是 x 的祖先。

贡献计算：

如果 y 是 x 的祖先，那么从 $root \to x$ 的路径上，位于 y 及其下方的点（即属于子树 y 的点）都会增加 v。这些点的数量恰好是 $depth[x] - depth[y] + 1$。

$$\text{Contribution} = v \times (depth[x] - depth[y] + 1) \tag{58}$$

由于查询点 x 的深度 $depth[x]$ 是随查询变化的，我们无法直接将 $(depth[x] - depth[y] + 1) \times v$ 整体存入数据结构。我们需要将其拆分为**“关于 x 的变量”**和**“关于 y 的变量”**：

$$\text{Contribution} = v \cdot depth[x] - v \cdot depth[y] + v \tag{59}$$

合并同类项：

$$\text{Contribution} = \underbrace{v \cdot (depth[x] + 1)}_{\text{变量与查询点相关}} - \underbrace{v \cdot depth[y]}_{\text{变量与修改点相关}} \tag{60}$$

为了维护上述拆分后的贡献，我们需要在 DFS 序上建立两个树状数组（BIT）：

1. **BIT1** (s_1)：维护修改值 v 的**区间增量**。
    - 修改子树 y 时：在 $st[y]$ 处 $+v$，在 $en[y]+1$ 处 $-v$。
2. **BIT2** (s_2)：维护 $v \cdot depth[y]$ 的**区间增量**。
    - 修改子树 y 时：在 $st[y]$ 处 $+(v \cdot depth[y])$，在 $en[y]+1$ 处 $-(v \cdot depth[y])$。

根据拆分公式，查询点 x 时，先求出 $st[x]$ 位置的前缀和：

$$\Delta Dis(x) = (depth[x] + 1) \cdot \text{sum1}(st[x]) - \text{sum2}(st[x]) \tag{61}$$

- `sum1(st[x])`：即此时路径上所有有效的 v 之和。
- `sum2(st[x])`：即此时路径上所有有效的 $v \cdot depth[y]$ 之和。

对于查询任意两点 a, b 之间的链和，我们依然使用 LCA 路径拆分公式：

$$\text{Ans} = \text{Path}(a) + \text{Path}(b) - \text{Path}(LCA) - \text{Path}(fa[LCA]) \tag{62}$$

其中每一项 $\text{Path}(i)$ 都由两部分组成：

1. **初始链和**：预处理 DFS 时求得的原始点权链和。
2. **动态增量**：即 $\Delta Dis(i) = (depth[i] + 1) \cdot \text{sum1}(st[i]) - \text{sum2}(st[i])$。

7.2 时间复杂度

动作	操作内容	复杂度
子树修改 (y, v)	`BIT1.update(st[y], v)`,`BIT2.update(st[y], v*dep[y])`（注意边界减回）	$O(\log n)$
路径求增量	`(dep[x]+1)*BIT1.query(st[x]) - BIT2.query(st[x])`	$O(\log n)$
链和查询 (a, b)	求 LCA，代入路径拆分公式计算 4 个点的 Path 值	$O(\log n)$

7.3 标准代码

```cpp
// 查询链和核心
ll query_path(int u) {
    return (ll)(dep[u] + 1) * sum1(be[u]) - sum2(be[u]);
}
// 最终答案
ans = query_path(a) + query_path(b) - query_path(lca) - query_path(f[lca]
```

```
[0]);
```